

Computer Vision Coding Lab

Exploring Digital Images and Colors with Python

Before You Begin

Google Colab + Python

This laboratory helps you understand digital images by experimenting with image data. Run and modify the short Python programs in Google Colab.

Important: This page is your activity guide. Copy the Python code into your Google Colab notebook and execute it there.

- Understand an image as numerical data.
- Explore RGB pixel values.
- Understand image dimensions and color channels.
- Observe what happens when RGB channels are changed.
- Explore RGB and HSV representations.
- Connect lighting and white balance to captured pixel values.

Progress:

Activity 1 — Your First Digital Image

Create a tiny grayscale image from a numerical array.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
image = np.array([
    [0, 50, 100, 150, 200],
    [50, 100, 150, 200, 250],
    [100, 150, 200, 250, 255],
    [150, 200, 250, 255, 200],
    [200, 250, 255, 200, 100]
])

print(image)
print("Shape:", image.shape)

plt.imshow(image, cmap='gray')
plt.axis('off')
plt.show()
```

Investigate

1. How many pixels are in the image?
2. What is the smallest value?
3. What is the largest value?
4. What does a value close to 0 represent?
5. What does a value close to 255 represent?

Your observations

- 3. pixel [2,4], [3,3], and [4,2] has the highest value of 255
- 4. Lower values represent a blacker pixel
- 5. Higher values represent a whiter pixel

Challenge: Change the array to create a simple pattern.

```
[
    [255, 255, 0, 255, 255],
    [255, 0, 255, 0, 255],
    [0, 255, 255, 255, 0],
    [255, 0, 255, 0, 255],
    [255, 255, 0, 255, 255]
]
```

This draws a diamond

Activity 2 — Let's Create RGB Pixels

An RGB pixel contains three numerical values.

```
import numpy as np

pixel = np.array([255, 0, 0])
print("RGB values:", pixel)
```

Predict: What color do you expect?

A fully red pixel since the red value is maxed, while green and blue are at 0

```
pixels = np.array([
    [255, 0, 0],
    [0, 255, 0],
    [0, 0, 255],
    [255, 255, 0],
    [255, 0, 255],
    [0, 255, 255],
    [255, 255, 255],
    [0, 0, 0]
])
print(pixels)
```

Which row represents white? Why?

the 7th row (second-last) represents white since all its RGB values are maxed

Which row represents black? Why?

The 8th row (last) represents black since all the RGB values are 0

Activity 3 — Build Your Own RGB Image

Arrange RGB pixels into rows and columns to create an image.

```
import numpy as np
import matplotlib.pyplot as plt

image = np.zeros((100, 100, 3), dtype=np.uint8)

image[0:50, 0:50] = [255, 0, 0]
image[0:50, 50:100] = [0, 255, 0]
image[50:100, 0:50] = [0, 0, 255]
image[50:100, 50:100] = [255, 255, 0]

plt.imshow(image)
plt.axis('off')
plt.show()
```

What does (100, 100, 3) mean?

this line configures the image, setting it to 100 height, 100 width, and 3 color channels

Why is there a 3?

The 3 in that line represent how many color channels there are. We need 3 for RGB

Which line creates yellow?

This line: `image[50:100, 50:100] = [255, 255, 0]`

Challenge: Create a 2×3 grid containing red, green, blue, cyan, magenta, and yellow.

```
import numpy as np
import matplotlib.pyplot as plt

image = np.zeros((100, 150, 3), dtype=np.uint8)

image[0:50, 0:50] = [255, 0, 0]    # Red
image[0:50, 50:100] = [0, 255, 0]  # Green
image[0:50, 100:150] = [0, 0, 255] # Blue

image[50:100, 0:50] = [0, 255, 255] # Cyan
image[50:100, 50:100] = [255, 0, 255] # Magenta
image[50:100, 100:150] = [255, 255, 0] # Yellow
```

Activity 4 — Become a Pixel Detective

Upload one of your own photographs to Google Colab.

```
from google.colab import files
from PIL import Image
import matplotlib.pyplot as plt

uploaded = files.upload()
filename = list(uploaded.keys())[0]

image = Image.open(filename).convert("RGB")

plt.imshow(image)
plt.axis("off")
plt.show()
```

Inspect its numerical representation:

```
import numpy as np

image_array = np.array(image)

print("Array shape:", image_array.shape)
print("Number of dimensions:", image_array.ndim)
```

If the output is **(720, 1280, 3)**, interpret it as height × width × color channels.

What was the shape of your image?

(1280, 1280, 3)

What do the three dimensions mean?

It means that the image has 3 dimensions of how the data of the image is organized. 1st dim being the rows (image height), 2nd dim being the cols (image width), and the 3rd dim being the RGB color channels

Activity 5 — What Does One Pixel Look Like?

```
y = 100
x = 200

pixel = image_array[y, x]
print("Pixel RGB:", pixel)
```

Change x and y and inspect different parts of your image.

Location (x, y)	RGB value	What color/area is it?
y=710, x=1130	[70 45 40]	Brown
y=1136, x=608	[212 221 230]	white
y=8, x=184	[81 63 51]	brown
y=131, x=769	[198 205 215]	blueish-grey
y=262, x=1001	[5 10 2]	black

Are pixels belonging to the same object exactly the same RGB values? Why or why not?

No, the pixels of an object don't have the same RGB value because external factors such as: lighting, shadows, reflections, texture, etc. affect it. So object won't have 1 uniform color value

Activity 6 — Separate the Colors

```
R = image_array[:, :, 0]
G = image_array[:, :, 1]
B = image_array[:, :, 2]

plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.imshow(R, cmap='gray')
plt.title("Red Channel")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(G, cmap='gray')
plt.title("Green Channel")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(B, cmap='gray')
plt.title("Blue Channel")
plt.axis("off")

plt.show()
```

Are the three channel images identical?

No, they all look different

Which channel appears brightest in some areas?

For me, the red channel appeared to be the brightest particularly in the face (the image was a picture of myself)

Why do the three channels contain different information?

Because different objects and surfaces reflect different amounts of red, green, and blue light.

Activity 7 — What Happens When We Remove a Color?

```
no_red = image_array.copy()
no_red[:, :, 0] = 0

plt.imshow(no_red)
plt.axis("off")
plt.show()
```

Repeat for green and blue by changing the channel index.

```
no_green = image_array.copy()
no_green[:, :, 1] = 0

plt.imshow(no_green)
plt.axis("off")
plt.show()

no_blue = image_array.copy()
no_blue[:, :, 2] = 0

plt.imshow(no_blue)
plt.axis("off")
plt.show()
```

Which removed channel produced the most noticeable change?

The image drastically changed when I removed the green channel. It made the image look pink

Did the objects themselves lose that color, or did we change their numerical representation?

No, the image itself did not change. We just changed the numerical representation of the image stored in image_array

Activity 8 — RGB Color Mixer

```
import numpy as np
import matplotlib.pyplot as plt

def show_color(r, g, b):
    color = np.zeros((100, 100, 3), dtype=np.uint8)
    color[:, :] = [r, g, b]
    plt.imshow(color)
    plt.axis("off")
    plt.title(f"RGB = ({r}, {g}, {b})")
    plt.show()

show_color(255, 100, 50)
```

Find RGB values resembling purple, orange, pink, dark green, light blue, brown, and gray.

Target	R	G	B
Purple	128	0	128
Orange	255	165	0
Pink	255	192	203
Dark green	0	100	0
Light blue	173	216	230

Brown	165	42	42
Gray	128	128	128

What happens when you increase all three values equally?

The color becomes brighter while maintaining the same hue

Activity 9 — RGB vs. HSV

```
import cv2
import matplotlib.pyplot as plt

hsv_image = cv2.cvtColor(image_array, cv2.COLOR_RGB2HSV)

print("RGB:", image_array[100, 100])
print("HSV:", hsv_image[100, 100])

H = hsv_image[:, :, 0]
S = hsv_image[:, :, 1]
V = hsv_image[:, :, 2]

plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.imshow(H, cmap='gray')
plt.title("Hue")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(S, cmap='gray')
plt.title("Saturation")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(V, cmap='gray')
```

```
plt.title("Value")  
plt.axis("off")  
  
plt.show()
```

How is HSV different from RGB?

HSV represent the color in a more human-intuitive way by structuring the color as: Hue, Saturation, and Value. Compared to RGB, which is harder for humans to immediately see what the color is just by looking at the RGB values

Which HSV component seems related to the type of color?

Hue (H)

Which component seems related to color strength?

Saturation (S)

Which component seems related to brightness?

Value (V)

Why might HSV be useful in a computer vision application?

HSV can make it easier to detect objects based on their color, because in RGB we have to consider all 3 values to determine the pixel color

Activity 10 — The White Balance Investigation

Use a photograph containing an object that should be white. Inspect several pixels on that object.

```
# Change these coordinates to a point on the white object  
y = 200  
x = 300  
  
print("RGB value:", image_array[y, x])
```

Think before you answer: If the object is physically white, should R, G, and B necessarily be exactly equal in a photograph?

What did your selected white-object pixels look like numerically?

[235, 238, 230]

Why might R, G, and B not be equal?

Through our eyes it might look white, but when we inspect its pixels, the RGB values are usually not equal. This is because when we took the picture, different factors such as lighting, texture, shadows, and image processing affected the image.

Overview

1. First Image

2. RGB Pixels

3. Build RGB

4. Pixel Detective

5. RGB Channels

6.

object will look yellowish or blueish, and the RGB values will also reflect the color cast

How does this connect to our earlier shadow experiment?

Back in assignment 1, a shadow is one of the external factors affecting the pixel values of an image. This is why when the white object was captured inside a dark place, it looked darker

Final Challenge — Create Your Own Computer Vision Color Experiment

Choose one experiment and implement it in Google Colab.

A. RGB Experiment

Create an image using at least five different RGB colors.

B. Pixel Experiment

Investigate at least ten pixels from an image.

C. Channel Experiment

Separate an image into R, G, and B channels.

D. HSV Experiment

Convert an image to HSV and investigate its channels.

E. Lighting Experiment

Photograph the same object under two lighting conditions and compare RGB values.

Required Submission

1. Google Colab notebook
2. Screenshot of your output
3. Python code
4. A 3–5 sentence explanation

What experiment did you choose?

E. Lighting Experiment

Key findings

I took a picture of an object with 2 different lightings: orange warm light and white cool light, whilst controlling other factors such as camera used, camera position, etc. When I inspected the same location for the two images and compared their RRGB values, i found that the pixel had greatly different values due to the different lighting conditions, with some values having a difference of 100-150 meaning that its greatly different

Complete: "A digital image is not simply a picture. It is..."

A digital image is not simply a picture, it is a numerical representation of pixels, where each pixel contains values that describe its color and brightness.

Save Your Activity Responses

Responses are saved temporarily in this browser. This page does not automatically send them to your instructor.

[Download My Answers](#)

[Clear Answers](#)

Computer Vision • Digital Images & Color Models • Coding Laboratory